

CI2

Cohort 3 Group 5

Members:

Amity Van Rooyen
Cassian Kanhukamwe
Dhruv Madan
Gilda Grimes
Jerry Anish
Matt Ritchie
Oakley Fiddler
Ruby Brown

Continuous Integration

Brownfield Development

CI Methods and Approaches

Continuous integration (CI) is a software development approach that enables the automatic verification of changes made by contributors to a shared codebase. Within the development of this project, the primary tools used to support continuous integration were Git, GitHub, and GitHub Actions. The main objective of adopting a CI approach was to ensure that code pushed to the project's primary branch could be integrated safely and consistently, thereby reducing the risk of integration issues such as compilation failures or broken functionality. The project implements a single centralised CI pipeline that automatically verifies the state of the project whenever changes are pushed to the main GitHub repository. This pipeline provides rapid feedback to developers and ensures that the shared codebase remains stable throughout development.

CI Pipeline Primary Features

- Automated build and compilation of the Java project
- Execution of unit tests, including a headless test configuration
- Generation of code coverage reports using JaCoCo
- Production of a compiled JAR file representing the current state of the project

The pipeline is triggered on every push to the primary branch of the GitHub repository. This ensures that all updates are verified immediately after integration, allowing developers to identify and address issues as early as possible.

Inputs to the CI pipeline include:

- The project's Java source code
- Gradle build configuration files
- Unit and headless test code
- Code coverage configuration

Justifications

The CI pipeline is deployed using GitHub Actions, an automated CI tool that executes workflows defined by a YAML configuration file stored within the repository. The presence of this file acts as an instruction set that GitHub Actions executes in response to Git events, such as pushes to the repository.

The selected tools and CI approach are well-suited to a Java-based, team-oriented development environment. Automated builds and tests reduce the need for manual verification and help ensure that contributions from multiple team members do not introduce regressions or break existing functionality.

Additionally, the use of code coverage reporting provides visibility into how thoroughly the project is tested, while the generation of a JAR file confirms that the project can be successfully compiled into a distributable format. Together, these features support reliable integration, improve maintainability, and promote confidence in the project's overall stability.

CI Infrastructure and Implementation

Workflow Structure and Execution

As outlined in the previous section, GitHub Actions was used as the primary continuous integration (CI) platform for this project. GitHub Actions provided the underlying framework required to implement a single CI pipeline, which was defined using a YAML workflow configuration file stored within the project repository.

This workflow file specifies the execution environment and the sequence of tasks that GitHub Actions should perform in order to verify each version of the project's code. The pipeline is configured to run in response to repository events, such as pushes and pull requests, ensuring that changes are automatically validated within a clean and consistent environment. The workflow and execution of the established CI pipeline is shown below.

- Checks out the repository's source code
- Sets up the required Java Development Kit (JDK)
- Executes Gradle build tasks
- Runs unit tests, including a headless test configuration
- Generates code coverage reports
- Builds and outputs a compiled JAR file

Build Automation and Tools

The project is developed using LibGDX, which is built on top of Gradle as its build system. Gradle was therefore used as the build automation tool within the CI pipeline, allowing all verification tasks—including building, testing, and coverage analysis—to be executed using standard Gradle commands (e.g. `gradlew build` and `gradlew test`). Using Gradle in both local development and the CI environment ensures consistency between local builds and automated CI builds, reducing the likelihood of discrepancies such as “works on my machine” issues. The feedback provided by the CI pipeline is shown below.

- Actions interface. For every pipeline run, developers are able to view:
- Whether the build succeeded or failed
- Detailed logs for each step of the workflow
- Unit test and verification results

In addition, the generated JAR file and reports (such as code coverage results) are stored as workflow artifacts, making them available for developers to download and review. This immediate and transparent feedback allows the development team to quickly identify and resolve any issues introduced by recent changes.

In summary, the CI infrastructure implemented for this project is simple yet effective, and well-matched to the project's scope and requirements. It provides automated verification, clear feedback, and reproducible builds for a Java-based, team-developed project. The current setup also leaves room for future enhancements, such as more advanced triggers or automated release pipelines, if required.